

Richiami hardware e alcuni meccanismi fondamentali

Antonio Pecchia

antonio.pecchia@unisannio.it

Dipartimento di Ingegneria
Laurea in Ingegneria Informatica
Sistemi Operativi



UNIVERSITÀ DEGLI STUDI
DEL SANNIO Benevento

RICHIAMI HARDWARE

Cap. 1: par. 1.3

Processore (CPU)

□ Registri:

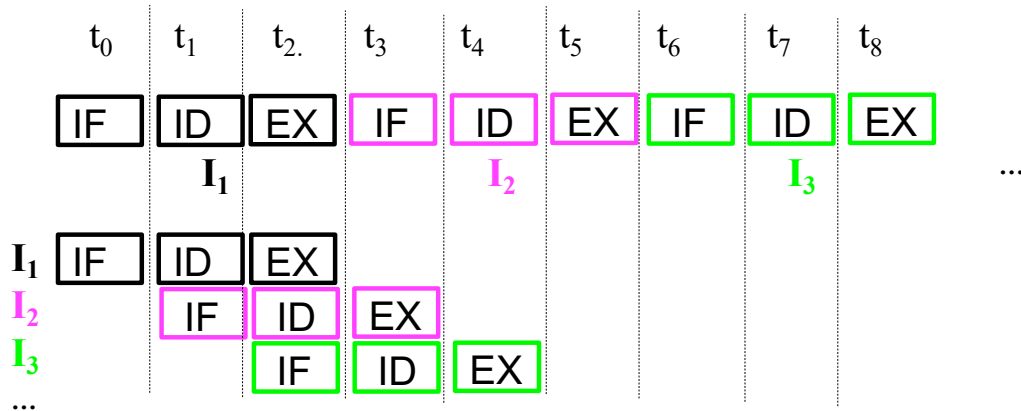
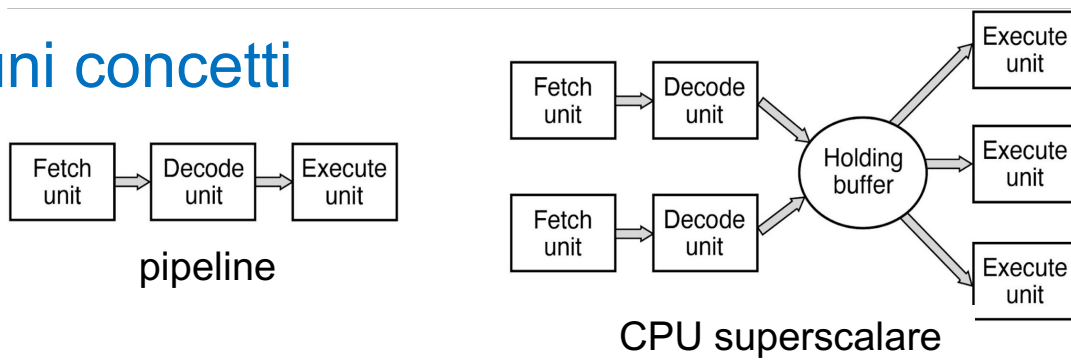
- registri generali / speciali (PC, SP, PSW*);
- tempo di **accesso**: frazione di clock;
- il *grosso* delle istruzioni sono fatte tramite registri:
 - **CISC** (Intel x86, PDP-11, Motorola 68000, ...) vs **RISC** (ARM, MIPS, SPARC...) .

***Program Status Word** (PSW): *codice di condizione, modalità (user o kernel), ...*

Basic x86 registers

Name	31	0	Use
EAX			GPR 0
ECX			GPR 1
EDX			GPR 2
EBX			GPR 3
ESP			GPR 4
EBP			GPR 5
ESI			GPR 6
EDI			GPR 7
	CS		Code segment pointer
	SS		Stack segment pointer (top of stack)
	DS		Data segment pointer 0
	ES		Data segment pointer 1
	FS		Data segment pointer 2
	GS		Data segment pointer 3
EIP			Instruction pointer (PC)
EFLAGS			Condition codes

Alcuni concetti



Altri aspetti

- Multicore, **hyperthreading**, ...

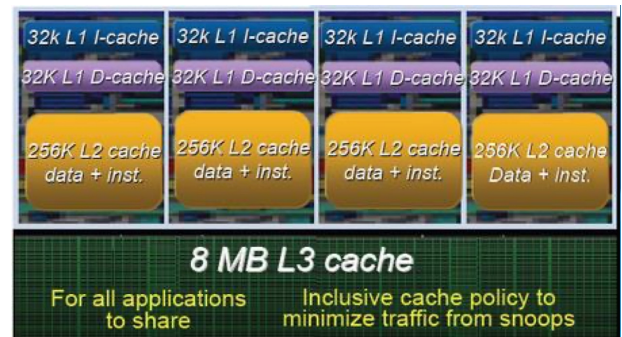
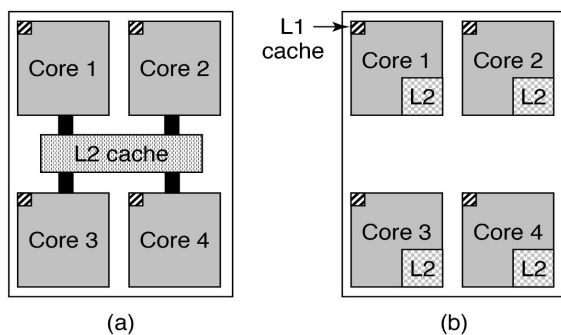


Figure 1-8. (a) A quad-core chip with a shared L2 cache.
(b) A quad-core chip with separate L2 caches.

Intel Core i7 caching
<https://web.cs.wpi.edu/>

Modello semplificato

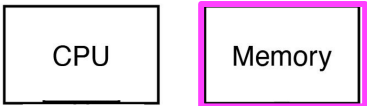
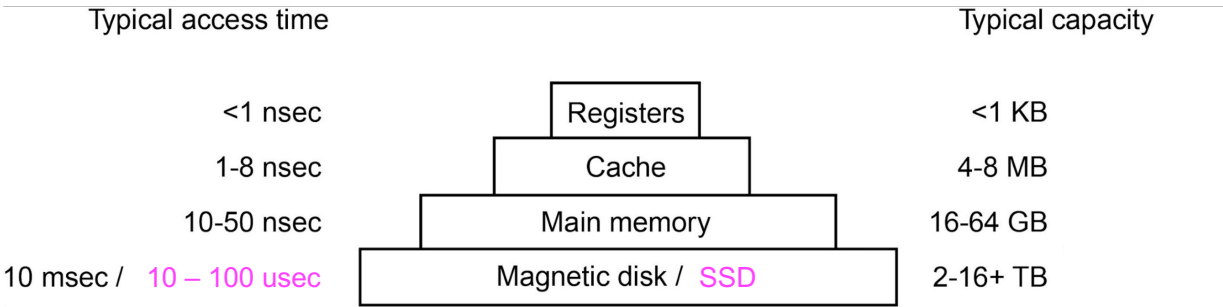


Figure 1-6. Some of the components of a simple personal computer.

Memoria principale – gerarchia di memoria

RAM (Random Access Memory)

10 ⁻³	0.001	milli	msec
10 ⁻⁶	0.000001	micro	usec
10 ⁻⁹	0.000000001	nano	nsec



Modello semplificato

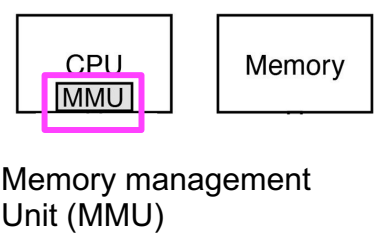


Figure 1-6. Some of the components of a simple personal computer.

Modello semplificato

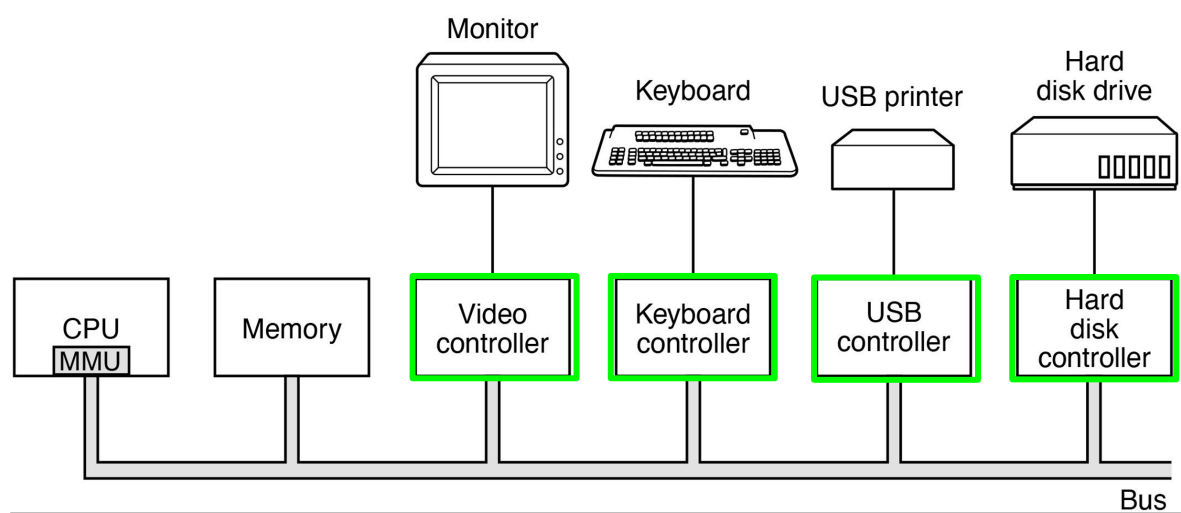


Figure 1-6. Some of the components of a simple personal computer.

I/O

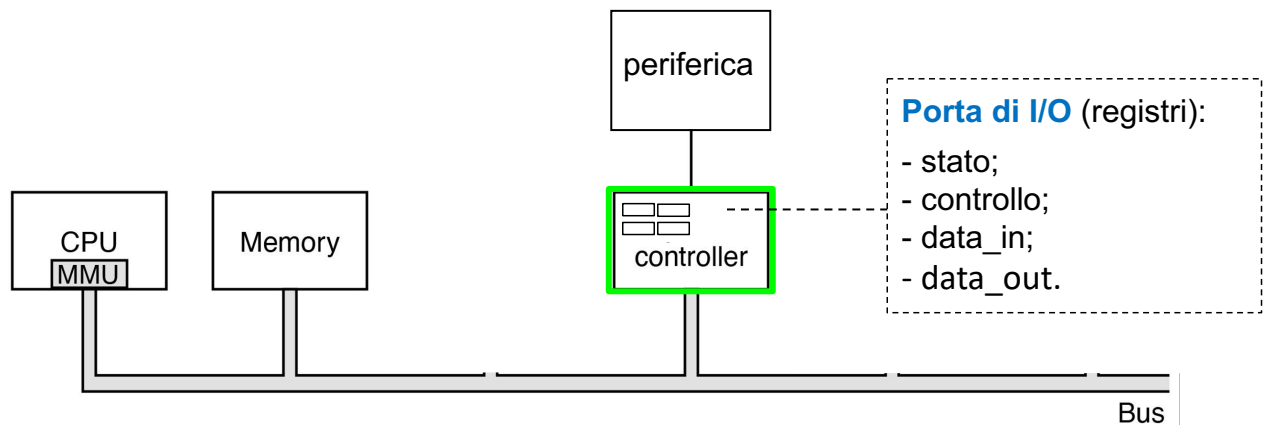


Figure 1-6. Some of the components of a simple personal computer.

I/O

□ Esempio di operazione I/O:

- il processo esegue una **system call**;
- il kernel invoca il **driver** appropriato;
- il driver avvia l'I/O.

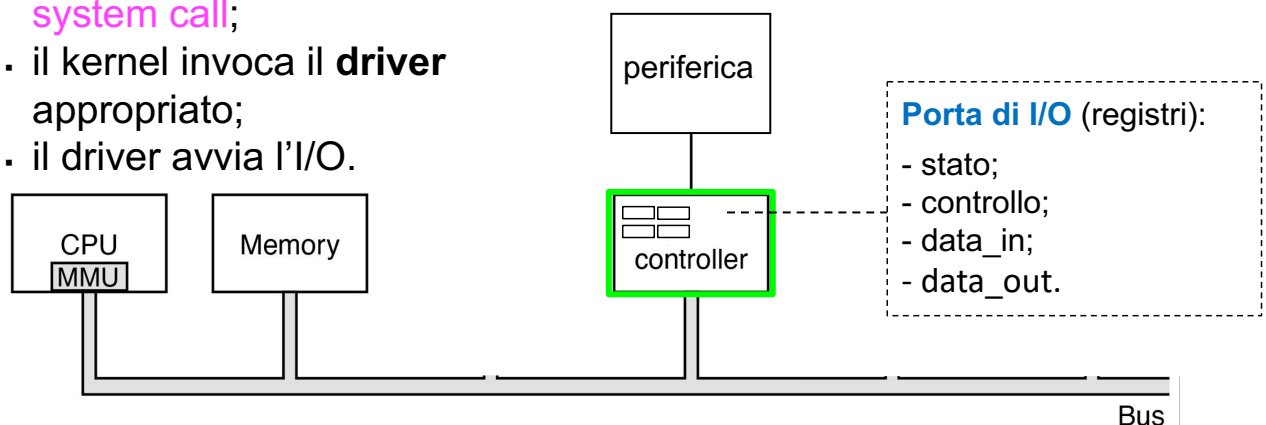
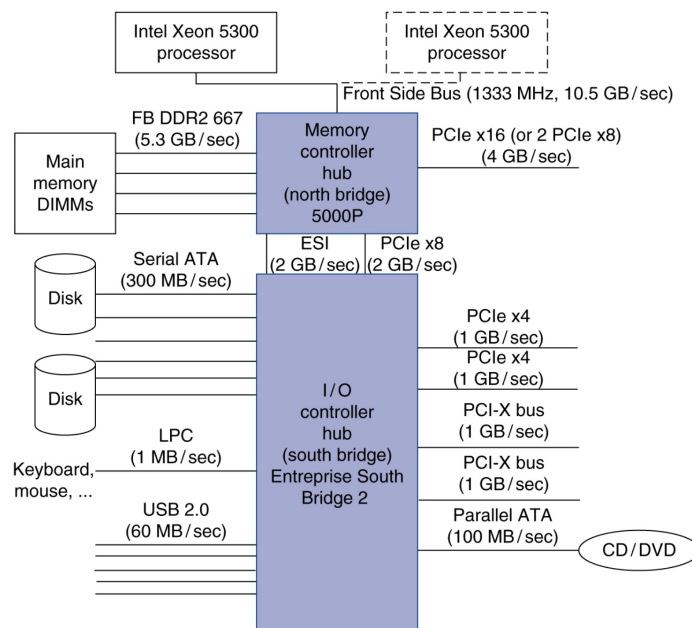


Figure 1-6. Some of the components of a simple personal computer.

I/O

- Accesso ai registri di I/O:
 - **port mapped** I/O (specifiche istruzioni asm `IN / OUT`);
 - **memory mapped** I/O (registri mappati nello spazio degli indirizzi del SO).
- Attesa / ottenimento delle risposte:
 - busy waiting – verifica periodica stato del dispositivo;
 - **interrupt**;
 - direct memory access (DMA):
 - flusso dati controllato da un chip DMA;
 - il chip DMA genera **interrupt**.

Typical x86 PC I/O System



ALCUNI MECCANISMI FONDAMENTALI

Interrupt: par. 1.3.4, 2.1.6, 5.1.5;
Chiamate di sistema: par. 1.6 (introduzione)
Avvio del computer: par. 1.3.6

Un po' di terminologia

- **System call**: richiesta "deliberata" di un servizio al SO, effettuata volutamente da un programma;
- **Eccezione**: evento (tipicamente *non* deliberato) generato durante l'esecuzione di una istruzione (per es., divisione per zero, errore di indirizzamento);
- **Interrupt hardware**: eventi hardware, **non causati** dal programma in esecuzione (per es., dispositivi I/O).

Un po' di terminologia

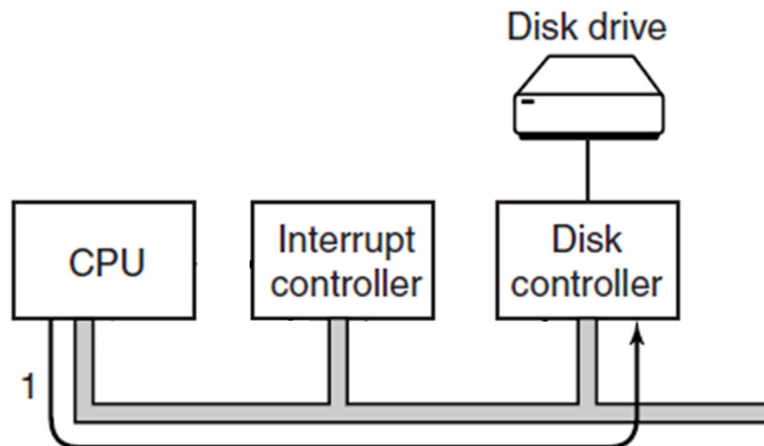
- A volte ci si riferisce a **syscall + eccezioni** (nel complesso) come **interrupt "software"**.
- Esistenza di una ulteriore nozione, **trap**, intesa in maniera diversa a seconda della trattazione:
 - qualsiasi forma di trasferimento di controllo al SO;
 - evento causato dal programma in esecuzione;
 - istruzione per passare in *kernel mode**;
 - le interruzioni software.

*Nozione utilizzata dal libro di testo (par 1.3.1)

Un po' di terminologia

- **Eventi asincroni:**
 - eventi **non causati** dall'istruzione in esecuzione:
 - causati da sorgente "esterna" (per es. dispositivi I/O).
- **Eventi sincroni:**
 - eventi **causati** dall'istruzione in esecuzione. Per es:
 - system call;
 - eccezioni rilevate dal processore (per es., divisione per zero, errore di indirizzamento).

Interrupt request



1) richiesta I/O

Figure 1-11. (a) The steps in starting an I/O device and getting an interrupt.

Interrupt request

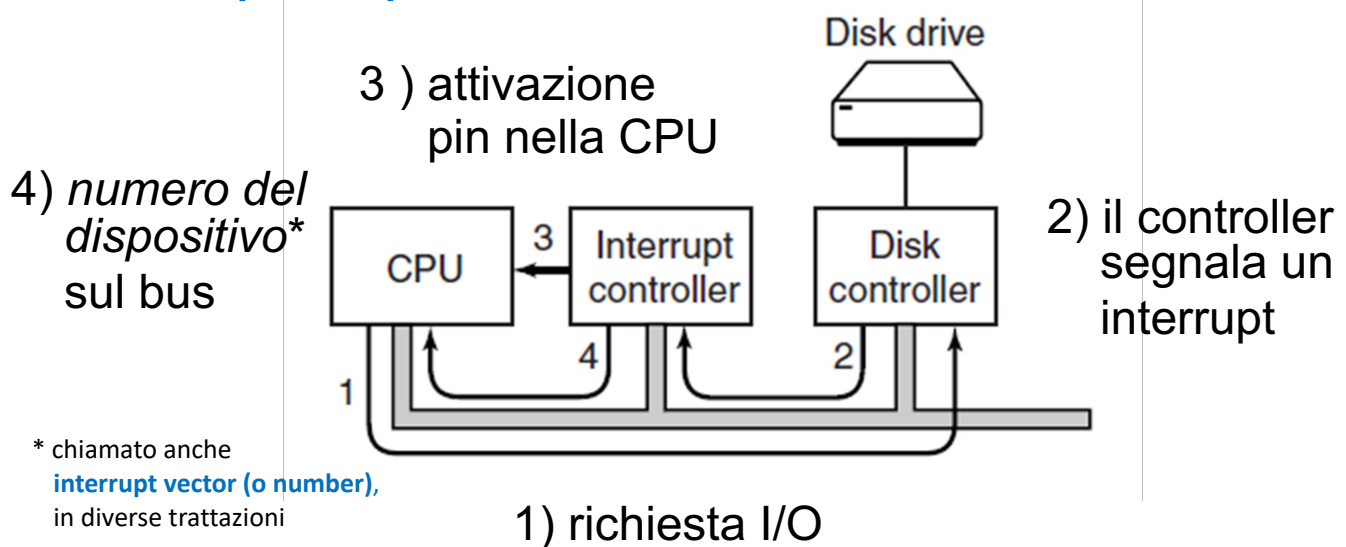


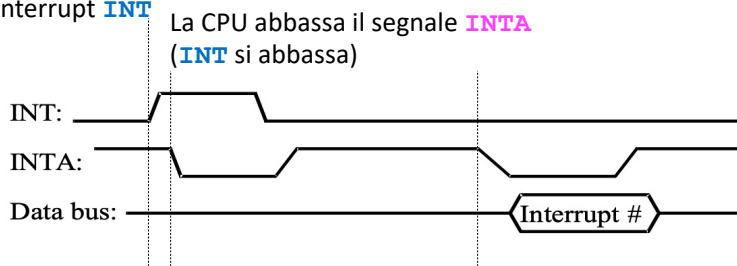
Figure 1-11. (a) The steps in starting an I/O device and getting an interrupt.

Interrupt controller: prima era un chip separato; ad oggi, è parte del **southbridge chipset** della scheda madre.

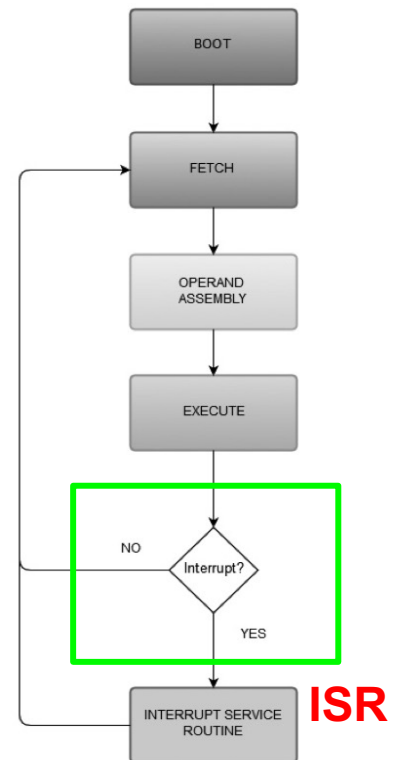
Sidenote

- Il ciclo del processore verifica la presenza di interruzioni.
- Esempio (Intel 80386) di protocollo CPU-interrupt controller (IC):

IC solleva la linea di interrupt **INT**



la CPU abbassa nuovamente il segnale **INTA**; IC mette l'**interrupt number** sul data bus



Interrupt handling

- La CPU salva il PC (più altri, *eventuali*, registri "critici", tipo SP e PSW) sul kernel stack;
- La CPU carica il PC con l'indirizzo dell'ISR (dalla [tabella dei vettori degli interrupt](#));

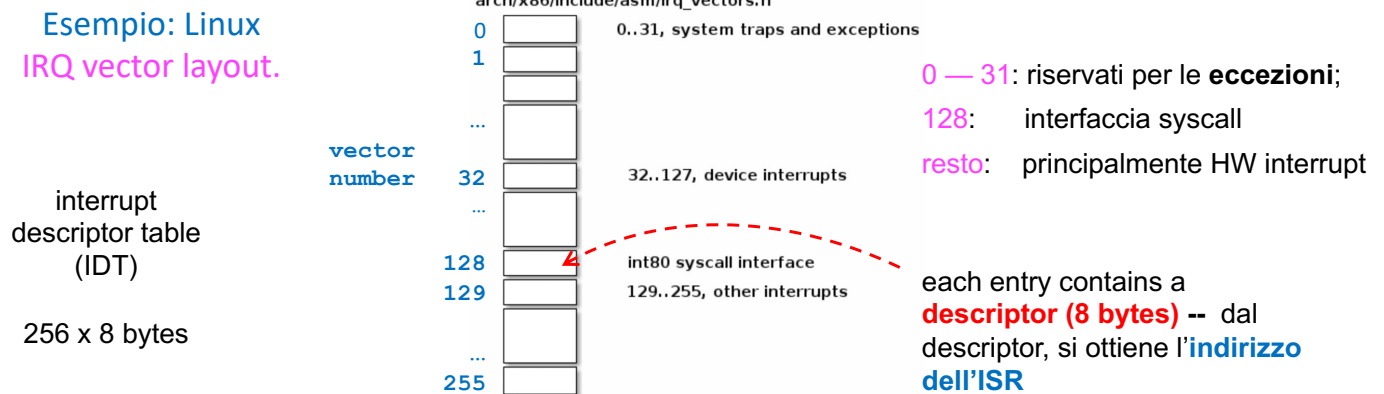
NOTE: I passi riportati in questa e nelle slide successive, rappresentano uno schema generale semplificato e a carattere indicativo.

I dettagli specifici variano a seconda del SO; alcuni step sono opzionali.

1, 2 sono parte del ciclo *hardware* dell'interrupt; la CPU 'passa' al kernel stack prima che il contenuto dei registri indicati al punto 1 vi venga salvato.

Interrupt handling

1. La CPU salva il PC (più altri, *eventuali*, registri "critici", tipo SP e PSW) sul kernel stack;
2. La CPU carica il PC con l'indirizzo dell'ISR (dalla [tabella dei vettori degli interrupt](#));



<https://linux-kernel-labs.github.io/refs/heads/master/lectures/interrupts.html>
https://github.com/torvalds/linux/blob/master/arch/x86/include/asm/irq_vectors.h

Interrupt handling

1. La CPU salva il PC (più altri, *eventuali*, registri "critici", tipo SP e PSW) sul kernel stack;
2. La CPU carica il PC con l'indirizzo dell'ISR (dalla [tabella dei vettori degli interrupt](#));
3. Salvataggio dei registri (*routine assembly*);
4. Impostazione di un nuovo stack (NOTA: step opzionale);
5. **Esecuzione della procedura di servizio dell'interrupt (ossia l'ISR);**

NOTE: Spesso, i registri al passo 3 più quelli salvati sullo stack al passo 1 sono posti nella **voce (entry) della tabella dei processi del processo corrente**; in alternativa, i registri continuano ad essere salvati sul kernel stack;

In generale, i passi 3 e 4 sono gli stessi per tutti gli interrupt.

Interrupt handling

1. La CPU salva il PC (più altri, *eventuali*, registri "critici", tipo SP e PSW) sul kernel stack;
2. La CPU carica il PC con l'indirizzo dell'ISR (dalla tabella dei vettori degli interrupt);
3. Salvataggio dei registri (*routine assembly*);
4. Impostazione di un nuovo stack (NOTA: step opzionale);
5. Esecuzione della procedura di servizio dell'interrupt (ossia l'ISR);
6. Lo **scheduler** decide quale processo eseguire come successivo;
7. La procedura in C ritorna al codice assembly (non è un "vero" step);
8. **Ripristino stato CPU** ('Avvio del nuovo processo corrente' – libro di testo);
9. Ritorno del controllo al processo (per es., istruzione x86 `iret`).

Interrupt handling

NOTE: Il **context switch** è al passo 8; tuttavia, non è detto il **c.s.** debba avvenire sempre.

Al passo 8, se il processo selezionato come 'successivo' (passo 6) è *differente* da quello corrente, è A) necessario salvare lo stato CPU del processo corrente nella **voce (entry) della tabella dei processi** del processo corrente e B) caricare –nei registri– lo stato CPU del processo successivo prelevandone le informazioni dalla **voce (entry) della tabella dei processi** del processo successivo.

6. Lo **scheduler** decide quale processo eseguire come successivo;
7. La procedura in C ritorna al codice assembly (non è un "vero" step);
8. **Ripristino stato CPU** ('Avvio del nuovo processo corrente' – libro di testo);
9. Ritorno del controllo al processo (per es., istruzione x86 `iret`).

NOTE: in caso di **context switch** il passo 9 verrà eseguito in *futuro* (se seguiamo la linea temporale del processo precedente).

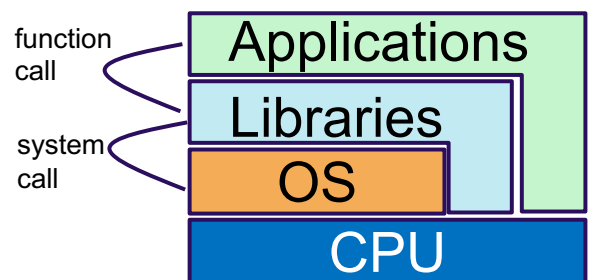
Interrupt handling (riepilogo)

1. La CPU salva il PC (più altri, *eventuali*, registri "critici", tipo SP e PSW) sul kernel stack;
2. La CPU carica il PC con l'indirizzo dell'ISR (dalla tabella dei vettori degli interrupt);
3. Salvataggio dei registri (*routine assembly*);
4. Impostazione di un nuovo stack (NOTA: step opzionale);
5. Esecuzione della procedura di servizio dell'interrupt (ossia l'ISR);
6. Lo scheduler decide quale processo eseguire come successivo;
7. La procedura in C ritorna al codice assembly (non è un "vero" step);
8. Ripristino stato CPU ('Avvio del nuovo processo corrente' – libro di testo);
9. Ritorno del controllo al processo (per es., istruzione x86 `iret`).

NOTE: Non sono stati indicati esplicitamente aspetti relativi allo **spazio degli indirizzi** (più avanti nel corso); il processo *successivo* può essere quello *corrente*.

System call

- Interfaccia che il SO offre ai programmi applicativi:
 - le system call sono tipicamente offerte tramite librerie;
 - per es., una funzione di libreria per ogni system call.
- *Gruppi* di system call formano **servizi**:
 - file system *service*;
 - process management *service*;
 - ...



Come avviene una system call

□ Esempi di invocazione tramite libreria:

- opzione 1: `syscall(1, 1, msg, sizeof(msg))`
- opzione 2: `write( 1, msg, sizeof(msg))`

□ Parametri (passati tramite registri):

convenzione registri

• `syscall number`

- `descrittore`
 - `buffer`
 - `# bytes`
- } altri parametri syscall
*fino a max 6**

x86_64

rax

rdi, rsi,
rdx, r10,
r8, r9

x86

eax

ebx, ecx,
edx, esi,
edi, ebp

*se ne servono altri, si userà un parametro come puntatore ad una struttura dati.

Come avviene una system call

`count = read(fd, buffer, nbytes);`

□ Inserimento parametri nei registri:

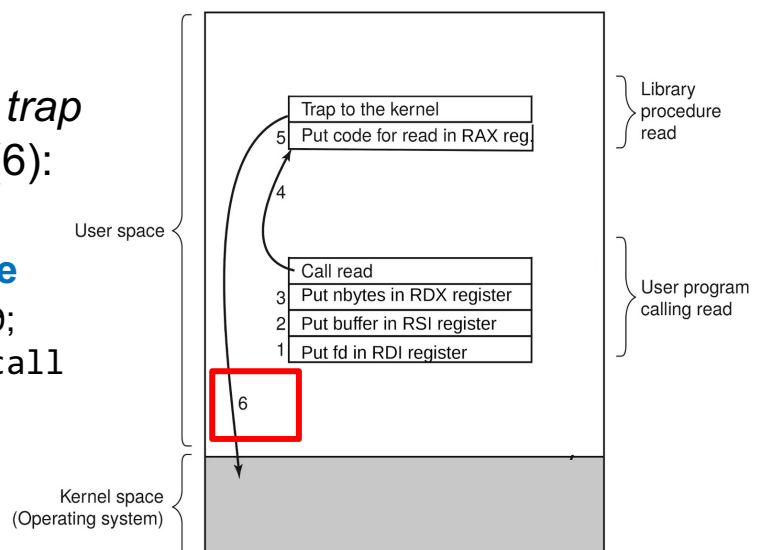
- passi 1-5, user space;

□ Esecuzione di una istruzione *trap* per entrare in **kernel space** (6):

Opzioni possibili*:

- fino a "ieri": una **interruzione software** (la 128) `int $0x80`;
- **oggi**: istruzione assembly `syscall` (in sistemi x86-64).

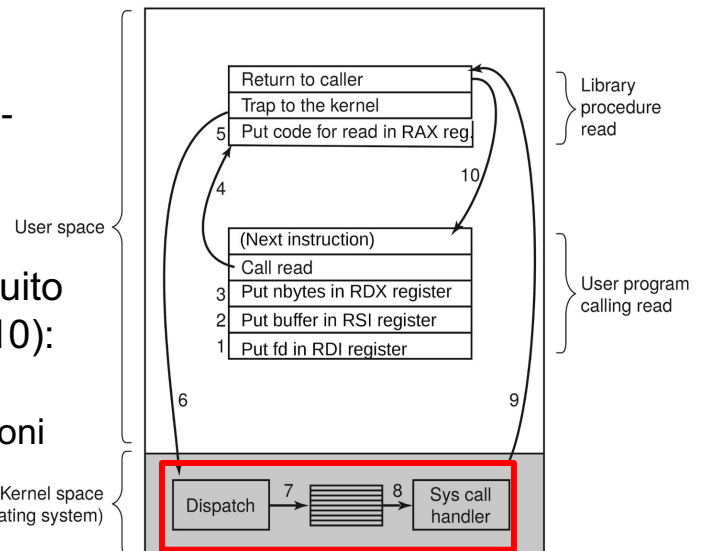
*in entrambi i casi si va in kernel mode.



Come avviene una system call

```
count = read(fd, buffer, nbytes);
```

- Si individua e si esegue il corretto **gestore** della chiamata di sistema:
 - 7: si esamina il contenuto in **rax** e si passa -tramite una **tabella** di puntatori- al gestore (o 'handler') della syscall;
 - **8: viene eseguito l'handler.**
- Il controllo può (**o non**) essere restituito alla libreria (9) e poi al chiamante (10):
 - NOTA: per es. la syscall potrebbe bloccare il chiamante (per es. operazioni I/O) → **context switch**:
 - 9-10 avverranno successivamente.



Eccezioni

Vector	Exception	Signal
0	Divide error	SIGFPE
1	Debug	SIGTRAP
2	NMI	None
3	Breakpoint	SIGTRAP
4	Overflow	SIGSEGV
5	Bounds check	SIGSEGV
6	Invalid opcode	SIGILL
7	Device not avail.	SIGSEGV
8	Double fault	SIGSEGV
9	Coproc.seg.ovr.	SIGFPE
10	Invalid TSS	SIGSEGV
11	Seg. not present	SIGBUS
12	Stack seg. fault	SIGBUS
13	General protect.	SIGSEGV
14	Page fault	SIGSEGV
15	Intel reserved	None
16	FP error	SIGFPE
17	Alignment check	SIGSEGV
18	Machine check	None
19	SIMD coproc.err	Depends
20	Virtualization exception	
21-31	Intel reserved	

- Molte delle eccezioni si traducono in **segnali***. Alcuni esempi:

- **SIGFPE** fatal arithmetic exception;
- **SIGILL** illegal instruction;
- **SIGBUS** bus error;
- **SIGSEGV** segmentation violation (out-of range address);
- ...

*esistono tanti altri segnali, anche non legati necessariamente alle eccezioni (per es., è possibile mandare un segnale ad un processo da prompt).

<https://faculty.cs.niu.edu/~hutchins/csci480/signals.htm>

Linux Interrupts: The Basic Concepts

[Intel® 64 and IA-32 Architectures Software Developer's Manual Vol 3A](#)

Eccezioni

- Il compito del SO è di mandare il segnale (se previsto) appropriato al processo corrente:
 - se il programma utente **ha specificato** un **exception handler**, il SO configura il programma utente in modo che possa invocare il suo handler;
 - in caso contrario, il SO **termina** ("uccide") il processo:
 - il **Page Fault** è un caso particolare (ci ritorneremo più avanti).

Handler (esempio minimale)

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>

void myhandler ( int signum ){
    printf( "Divide by zero (handler)\n" );
    exit(1);
}

int main( int argc, char** argv){

    signal( SIGFPE, myhandler );

    int a=10;
    int b=0;
    int c=a/b;

    return 0;
}
```

Esecuzione **senza** handler

\$./divZero

Eccezione in virgola mobile (core dump creato)

Esecuzione **con** handler

\$./divZero_handled

Divide by zero (handler)

Avvio del computer (firmware BIOS*)

- ❑ Basic Input Output System (BIOS):
 - **firmware di sistema** (su memoria flash della scheda madre).
- ❑ **Prima fase:**
 - all'avvio del computer, la CPU legge da un indirizzo mappato sulla flash;
 - esecuzione codice del BIOS;
 - rilevamento/inizializzazione risorse (RAM, controller interrupt, bus PCI);
 - set up firmware per alcuni servizi critici (low-level I/O).

*Oggi sostituito da UEFI (Unified Extensible Firmware Interface).

Avvio del computer (firmware BIOS*)

- ❑ **Seconda fase:**
 - il BIOS determina il dispositivo di avvio e ne legge/esegue il primo settore, detto Master Boot Record (**MBR**);
 - l'MBR contiene un programma che determina la "partizione attiva" e legge un **secondo bootloader**:
 - lettura/avvio del SO dalla partizione;
 - recupero della configurazione HW dal BIOS + eventuale installazione nuovi driver (se necessari);
 - caricamento driver nel kernel, creazione strutture dati, avvio processi background, **presentazione di login o della GUI.**

*Oggi sostituito da UEFI (Unified Extensible Firmware Interface).